

PSEUDOCODE WRITING TASKS

Computer Vision Pipelines Using OpenCV

QUESTION 4: Vehicle Counting System Using Video

Aim:

To detect, track, and count vehicles crossing a predefined region of interest (ROI) without double counting.

Pseudocode:

```
BEGIN

1. Initialize video capture using cv2.VideoCapture().
2. Load a trained vehicle detection model.
3. Initialize an object tracker.
4. Define the Region of Interest (ROI) and counting line.
5. Set vehicle_count = 0 and create an empty set counted_ids.

6. WHILE the video is running:
    a. Read the current frame using cap.read().
    b. IF frame is not available, BREAK.
    c. Resize the frame to reduce processing time.
    d. Detect vehicles in the current frame.
    e. FOR each detected vehicle:
        i. Get its bounding box.
        ii. Calculate the center point.
        iii. Assign or update a unique vehicle ID.
    f. Update the tracker using the detected positions.
    g. FOR each tracked vehicle:
        IF the vehicle crosses the counting line AND
        its ID is not in counted_ids:
            vehicle_count = vehicle_count + 1
            Add vehicle ID to counted_ids.
    h. Draw bounding boxes around detected vehicles.
    i. Draw vehicle IDs and the counting line.
    j. Display vehicle_count on the frame.
    k. Display the frame using cv2.imshow().
    l. IF the ESC key is pressed, BREAK.

7. Release the video using cap.release().
8. Close all windows using cv2.destroyAllWindows().

END
```

Important OpenCV Functions / Concepts:

- `cv2.VideoCapture()` – reads video input.
- `cv2.rectangle()` – draws bounding boxes around vehicles.
- `cv2.line()` – draws the counting line.

- `cv2.putText()` – displays vehicle IDs and count.
- `cv2.imshow()` – displays the processed video.

Justification: Unique vehicle IDs are stored in `counted_ids` so that the same vehicle is not counted more than once.

QUESTION 13: Video Stabilization Pipeline

Aim:

To reduce unwanted camera movement and jitter by estimating motion between consecutive frames and applying a corrective transformation.

Pseudocode:

```
BEGIN

1. Open the input video using cv2.VideoCapture().
2. Read the first frame.
3. Convert the first frame to grayscale.
4. Store it as previous_frame.

5. WHILE the video is running:
    a. Read the current frame.
    b. IF frame is not available, BREAK.
    c. Convert the current frame to grayscale.
    d. Detect strong feature points in previous_frame
       using cv2.goodFeaturesToTrack().
    e. Track these points in the current frame using
       cv2.calcOpticalFlowPyrLK().
    f. Remove invalid or missing feature points.
    g. Estimate the motion between the two frames.
    h. Calculate the transformation matrix.
    i. Apply the inverse transformation to the current
       frame using cv2.warpAffine().
    j. Crop or resize the stabilized frame to remove borders.
    k. Display the stabilized frame.
    l. Set current_frame as previous_frame.
    m. IF the ESC key is pressed, BREAK.

6. Release the video capture.
7. Close all OpenCV windows.

END
```

Important OpenCV Functions / Concepts:

- `cv2.goodFeaturesToTrack()` – detects strong feature points.
- `cv2.calcOpticalFlowPyrLK()` – tracks feature movement.
- `cv2.estimateAffinePartial2D()` – estimates the transformation.
- `cv2.warpAffine()` – applies the corrective transformation.

- cv2.imshow() – displays the stabilized video.

Justification: Motion is estimated from consecutive frames and the opposite transformation is applied to reduce camera jitter while preserving the scene.

QUESTION 1: Real-Time Face Detection and Recognition Pipeline

Aim:

To detect and recognize multiple faces from a live camera stream and display their names in real time.

Pseudocode:

```
BEGIN

1. Load a pre-trained face detection model.
2. Load a pre-trained face recognition dataset/model.
3. Open the camera using cv2.VideoCapture(0).

4. WHILE the camera is running:
    a. Capture a frame using cap.read().
    b. IF frame is not available, BREAK.
    c. Resize the frame for faster processing.
    d. Convert the frame to grayscale.
    e. Detect all faces in the frame.
    f. FOR each detected face:
        i. Extract the face ROI.
        ii. Preprocess the face.
        iii. Compare face features with the pre-trained dataset.
        iv. Find the best matching identity.
        v. IF confidence is acceptable:
            label = recognized person's name
        ELSE:
            label = "Unknown"
        vi. Draw a bounding box around the face.
        vii. Display the name and confidence.
    g. Display the processed frame.
    h. IF the ESC key is pressed, BREAK.

5. Release the camera.
6. Destroy all OpenCV windows.

END
```

Important OpenCV Functions / Concepts:

- cv2.VideoCapture() – captures live camera frames.
- cv2.cvtColor() – converts the image to grayscale.
- cv2.rectangle() – draws face bounding boxes.
- cv2.putText() – displays the recognized name.
- cv2.imshow() – displays the result in real time.

Justification: The system processes each frame continuously and uses a loop for multiple faces, allowing real-time recognition with minimal delay.

QUESTION 12: Face Detection with ROI Extraction

Aim:

To detect multiple faces in an image, extract each face as a Region of Interest (ROI), and save the cropped faces without redundant processing.

Pseudocode:

```
BEGIN

1. Read the input image using cv2.imread().
2. IF the image is not found:
    Display an error message and STOP.
3. Convert the image to grayscale.
4. Load a pre-trained face detector.
5. Detect all faces in the image.
6. Initialize face_number = 1.

7. FOR each detected face:
    a. Get x, y, width and height.
    b. Check whether this face has already been processed.
    c. IF the face is not already processed:
        i. Extract the face ROI:
            face = image[y:y+h, x:x+w]
        ii. Resize the cropped face if required.
        iii. Save the face as face_1.jpg, face_2.jpg, etc.
        iv. Draw a bounding box on the original image.
        v. Increase face_number by 1.

8. Display the image with detected face boxes.
9. Display the total number of detected faces.
10. Close all OpenCV windows.

END
```

Important OpenCV Functions / Concepts:

- cv2.imread() – reads the input image.
- cv2.cvtColor() – converts the image to grayscale.
- detectMultiScale() – detects faces.
- Array slicing – extracts the face ROI.
- cv2.imwrite() – saves cropped face images.
- cv2.rectangle() – draws bounding boxes.

Justification: Each detected face is extracted as a separate ROI and saved independently. Tracking processed regions prevents redundant processing.

QUESTION 2: OCR Pipeline for Document Processing

Aim:

To process multiple document images, improve image quality, segment text regions, and extract recognized text.

Pseudocode:

```
BEGIN

1. Get all document image filenames from the input folder.

2. FOR each image in the folder:
    a. Read the image using cv2.imread().
    b. IF the image cannot be read, CONTINUE to the next image.
    c. Resize the image if necessary.
    d. Convert the image to grayscale.
    e. Remove noise using GaussianBlur() or medianBlur().
    f. Improve contrast using histogram equalization or CLAHE.
    g. Apply thresholding to separate text from the background.
    h. Apply morphological opening/closing to remove noise
       and connect text regions.
    i. Detect text regions using contours.
    j. Extract each detected text ROI.
    k. Pass text regions to an OCR engine such as Tesseract.
    l. Store the recognized text.
    m. Draw bounding boxes around detected text regions.
    n. Display or save the processed image.

3. Combine or store the recognized text from all images.
4. Display the final extracted text.

END
```

Important OpenCV Functions / Concepts:

- `cv2.imread()` – reads document images.
- `cv2.cvtColor()` – converts images to grayscale.
- `cv2.GaussianBlur()` / `cv2.medianBlur()` – removes noise.
- `cv2.equalizeHist()` / CLAHE – improves contrast.
- `cv2.threshold()` – separates text from background.
- `cv2.morphologyEx()` – performs morphological filtering.
- `cv2.findContours()` – detects text regions.
- `cv2.rectangle()` – highlights text regions.

Justification: Preprocessing is important because document images may be noisy and have low contrast. Batch processing allows all images in the folder to be processed automatically.